

0.1 Motivation

A water distribution network, a power grid, a plant process line and a construction schedule are frequently the same kind of directed dependency structure, asked three different questions: does the system keep functioning when a component fails, how much can it carry, and when does a dependent sequence of activities finish. Reliability computation, flow analysis and schedule analysis answer these three questions separately, each with its own methods, and each typically consuming a single precise number for what it does not know exactly about its inputs, a component failure probability, an edge capacity, an activity duration.

No tool or method combines all three of these analyses on one network with any of them propagating interval or probability-box uncertainty natively. Huang, Huang and Lin’s exact project-reliability method pairs reliability with schedule but not flow, computing the probability that a project completes within both a time and a budget constraint from discrete, point-valued activity states [1]. Tong’s Georgia Tech dissertation pairs reliability with flow but not schedule, using a point-valued probability propagation method throughout [2]. Running several kinds of analysis against one shared network object is itself not new: **pandapower**, an open-source power-system tool, runs five different analyses, power flow, optimal power flow, state estimation, topological search and short-circuit calculation, against one shared graph of buses and lines [3]. None of these three, and nothing else found in a systematic search, combines reachability reliability, capacitated flow and activity scheduling on one network representation with native imprecision propagation across the combination. Chapter 2 surveys this landscape in full.

This thesis develops a framework that closes that gap: one network representation supporting all three analyses, native interval and probability-box propagation wherever the underlying computation supports it, and delivery through a local, no-code interface rather than a script. Figure 1 sketches the framework this way, top to bottom: the network representation and its decomposition strategy, the propagation algorithms the three toolkits share a common structure through, the point at which each toolkit’s imprecision support actually stops, reliability carrying intervals and probability boxes, schedule carrying intervals only, flow carrying neither yet, and the package and interface the whole framework is delivered through.

0.2 Objectives

The objectives of this thesis are summarised as follows:

- To develop a single directed-graph representation of a complex process, general enough that reachability reliability, capacitated flow and activity scheduling are each a reading of the same object rather than three separate models.

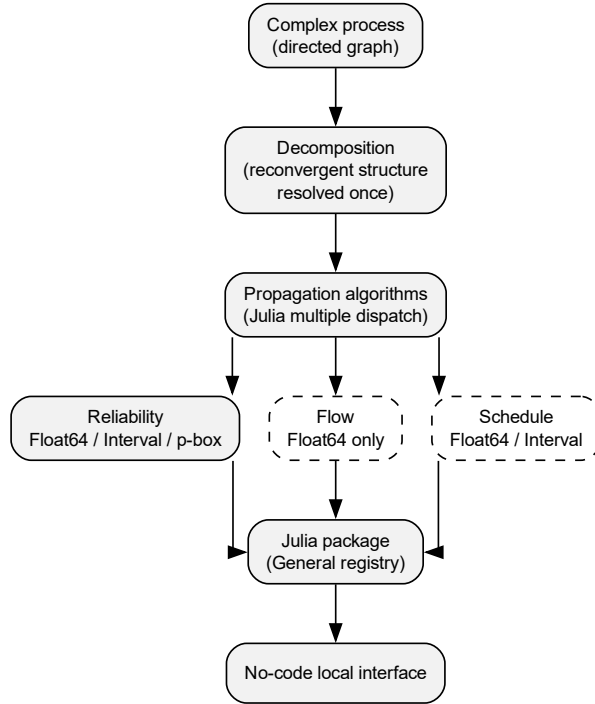


Figure 1: The framework as a layered toolkit. Contribution 1 is the network representation and its decomposition; Contribution 2 is the shared propagation algorithms; Contribution 3 is the interval and probability-box support each toolkit actually carries, full support in reliability, interval only in schedule, neither yet in flow; Contribution 4 is the interface; Contribution 5 is the package the interface is built on.

- To develop a decomposition strategy that resolves reconvergent structure, the redundancy that makes each of the three analyses expensive, once per query rather than once per analysis.
- To develop propagation algorithms, sharing one code path specialised by Julia’s multiple dispatch, that serve reliability, flow and schedule analysis alike without three independently maintained implementations.
- To extend those algorithms to intervals and probability boxes wherever the underlying computation supports it, and to state plainly, not elide, the domains and the specific value types where it currently does not.
- To deliver the framework as a documented, tested Julia package, published to the Julia General package registry.
- To deliver the framework through a local, no-code interface, built on that same package, that asks its user for files and clicks rather than code.

0.3 Contributions

The main contributions of this thesis are summarised as follows:

- **Contribution 1:** a unified network representation and a decomposition strategy for it, developed together, so that a reconvergent structure identified once is available to every analysis run against the network rather than rediscovered per analysis.
- **Contribution 2:** propagation algorithms, generic across value type through Julia’s multiple dispatch, that compute reliability, flow and schedule analysis from one shared code structure rather than three independently maintained ones.
- **Contribution 3:** native propagation of interval and probability-box uncertainty through the reliability computation, and of interval uncertainty, not yet probability-box, through the schedule computation. The flow computation shares neither and remains point-valued. Each of these three boundaries is stated plainly here, not written around.
- **Contribution 4:** a local, no-code interface to the framework, so that running an analysis does not require writing or reading code.
- **Contribution 5:** the framework delivered as a ready-to-use Julia package, published to the Julia General package registry, and used directly as the server backend behind Contribution 4’s own interface rather than as a separate research prototype alongside it.

0.4 Thesis Structure

Chapter 2 surveys the literature this thesis positions itself against: the separate literatures for reliability, flow and schedule analysis, how each represents what it does not know precisely, the closest existing tools and methods that combine more than one of the three analyses, and the languages and software delivery patterns already established in adjacent engineering fields.

Chapter 3 develops the network representation itself, a directed graph general enough to support all three readings of the same structure.

Chapter 4 describes how a network definition is parsed, validated and turned into that graph object.

Chapter 5 develops the decomposition strategy that resolves reconvergent structure once, the shared foundation every analysis toolkit that follows depends on.

Chapter 6 develops the reliability toolkit: exact source-to-node reachability, computed by message passing, generic across point-valued, interval and probability-box inputs.

Chapter 7 develops the flow toolkit: exact capacitated flow, and states its current point-valued boundary plainly.

Chapter 8 develops the schedule toolkit: the critical path method extended to interval durations, generic across the same value types as the reliability toolkit.

Chapter 9 discusses the framework’s Julia-specific implementation, including the server backend the framework’s own interface is built on, and the framework’s delivery as a package in the Julia General registry, without repeating the algorithmic content already developed in the chapters above.

Chapter 10 develops the no-code local interface, positioned against the delivery patterns Chapter 2 surveys.

Chapter 11 applies the complete framework to a single system, running reliability, flow and schedule analysis on it through the interface Chapter 10 develops, and closes the thesis.

0.5 Publications

According to the results of the research reported in this thesis, the following papers have been published, submitted, or are in preparation:

- Ohiani, T. and Patelli, E. (2023). “The Information Propagation Method for Efficient Network Reliability Analysis.” *2023 7th International Conference on System Reliability and Safety (ICSRS)*, 580–584.
- Ohiani, T. and Patelli, E. Journal paper submitted to *Reliability Engineering & System Safety* (in preparation).

- Conference paper on the framework’s no-code interface (planned).
- Conference paper for JuliaCon (planned).

0.6 Summary

Reliability, flow and schedule analysis are usually computed separately, each on its own network model, each with its own answer to what it does not know precisely about its inputs. No existing tool, method or doctoral thesis combines all three on one network representation with native imprecision propagation across the combination, and none delivers that combination without asking its user to write code. This thesis develops a framework that does both, delivered as a Julia package and through a local, no-code interface built on it.

Bibliography

- [1] D.-H. Huang, C.-F. Huang, and Y.-K. Lin, “Exact project reliability for a multi-state project network subject to time and budget constraints,” *Reliability Engineering & System Safety*, vol. 195, p. 106 744, 2020. DOI: 10.1016/j.ress.2019.106744
- [2] Y. Tong, “New approaches for modeling and reliability assessment of infrastructure flow networks,” Ph.D. dissertation, Georgia Institute of Technology, 2021.
- [3] L. Thurner et al., “Pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems,” *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018. DOI: 10.1109/TPWRS.2018.2829021